



Exercices

Des interfaces aux Streams API

Programmation orientée objet et fonctionnelle en Java

RABOANARY Heriniaina Andry

Public : L2 Informatique

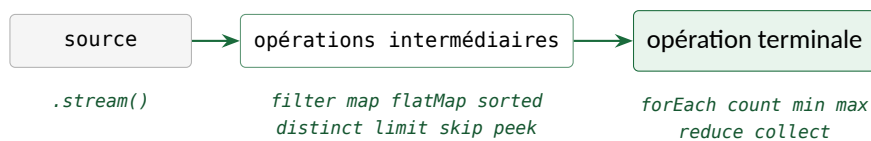
Document lié : polycopié « Des interfaces aux Streams API »

Environnement : Java 8 ou ultérieure

Année universitaire : 2025-2026

Consignes

- Chaque exercice se traite dans **une seule classe**, nommée Exercice1, Exercice2, etc., contenant une méthode main.
- Compilation et exécution : `javac Exercice1.java` puis `java Exercice1`. Sauf mention contraire, aucune boucle `for` ni `while` n'est autorisée là où un Stream peut faire le travail.
- Choisir soi-même les données de départ (noms, notes, produits, étudiants) : elles doivent être assez variées pour que chaque question ait un résultat visible.
- Les questions d'un même exercice sont indépendantes : une question bloquée n'empêche pas de traiter les suivantes.

Rappel — le pipeline Stream

Sans opération terminale, rien ne s'exécute — et aucune erreur n'apparaît.

1. Exercice 1 — Interface fonctionnelle et lambdas

Objectif — écrire une interface fonctionnelle et l'implémenter par quatre lambdas.

Créer une interface fonctionnelle `Operation` contenant une méthode :

```
int appliquer(int a, int b);
```

Créer ensuite les lambdas suivantes : addition, soustraction, multiplication, maximum.

Résultat attendu — pour `a = 7` et `b = 3` : 10, 4, 21, 7.

2. Exercice 2 — Streams sur une liste de chaînes

Objectif — enchaîner `filter`, `map`, `sorted` et `count` sur des chaînes.

Créer une liste de noms. Avec les Streams API :

1. afficher tous les noms ;
2. afficher les noms qui commencent par A ;
3. transformer tous les noms en majuscules ;
4. trier les noms par ordre alphabétique ;
5. compter les noms ayant plus de 5 caractères.

Résultat attendu — un seul parcours par question, sans aucune boucle `for`.

3. Exercice 3 — Streams sur une liste de notes

Objectif — utiliser les opérations terminales de comptage et d'agrégation.

Créer une liste de notes. Avec les Streams API :

1. afficher les notes supérieures ou égales à 10;
2. calculer le nombre d'admis;
3. calculer la moyenne;
4. afficher la meilleure note;
5. trier les notes dans l'ordre décroissant.

Résultat attendu — la meilleure note doit être affichée sans les mots `Optional[...]`.

4. Exercice 4 — Classe Produit et Collectors

Objectif — appliquer les Streams à une collection d'objets métier.

Créer une classe `Produit` contenant : `nom`, `categorie`, `prix`. À l'aide des Streams API :

1. afficher les produits de prix supérieur à 10 000;
2. trier les produits par prix décroissant;
3. calculer le prix moyen;
4. regrouper les produits par catégorie;
5. trouver le produit le plus cher.

Résultat attendu — le regroupement par catégorie doit s'obtenir en une seule instruction.

5. Exercice 5 — Synthèse : gestion d'étudiants

Objectif — reprendre l'étude de cas du cours de façon autonome.

Créer une classe `Etudiant` contenant : `matricule`, `nom`, `niveau`, `moyenne`. À l'aide des Streams API :

1. afficher les étudiants admis;
2. afficher les étudiants non admis;
3. trier les étudiants par moyenne décroissante;
4. calculer la moyenne générale;
5. regrouper les étudiants par niveau;
6. afficher le meilleur étudiant.

Résultat attendu — admis et non admis obtenus par un seul `partitioningBy`.

Avant de rendre

Vérifications à faire soi-même

1. Le programme **compile** sans avertissement et **s'exécute** jusqu'au bout.
2. Chaque question produit un affichage : un pipeline sans opération terminale ne fait rien, et ne signale rien.
3. Aucun `Optional[...]` ne subsiste dans l'affichage.
4. Aucun Stream n'est stocké dans une variable puis réutilisé.
5. Les noms de variables et de lambdas sont explicites : `note`, `produit`, `etudiant` plutôt que `x` ou `p`.

Les exercices 1 à 3 sont à traiter en priorité ; les exercices 4 et 5 réinvestissent les mêmes opérations sur des objets métier.

Aide-mémoire

Opérations intermédiaires — rendent un nouveau Stream

<code>filter</code>	conserve les éléments vérifiant une condition
<code>map</code>	transforme chaque élément
<code>flatMap</code>	aplatit des collections imbriquées
<code>sorted</code>	trie (comparateur optionnel)
<code>distinct</code>	supprime les doublons
<code>limit / skip</code>	garde / ignore les n premiers
<code>peek</code>	observe sans transformer (débogage)

Opérations terminales — déclenchent le traitement

<code>forEach</code>	traite chaque élément, ne rend rien
<code>count</code>	un long
<code>min / max</code>	un <code>Optional</code> , à débiller avec <code>ifPresent</code> ou <code>orElse</code>
<code>reduce</code>	une valeur unique agrégée
<code>collect</code>	une collection ou un résultat structuré

Collectors — `java.util.stream.Collectors`

<code>toList / toSet</code>	une <code>List</code> / un <code>Set</code>
<code>joining(", ")</code>	une chaîne concaténée
<code>counting()</code>	un effectif
<code>averagingDouble</code>	une moyenne
<code>groupingBy</code>	une <code>Map</code> clé → liste
<code>partitioningBy</code>	une <code>Map</code> à deux clés <code>true / false</code>
<code>summarizingDouble</code>	min, max, moyenne, somme, effectif